

Projeto Final Cobol - Modernização de um Sistema Legado*

Vitor Leite Setragni¹

Resumo

Este projeto tem como objetivo modernizar um sistema legado de cadastro de clientes da Cooperativa Financeira Alfa. A solução mantém o processamento principal em COBOL, preservando a lógica do sistema legado. Para facilitar o acesso do usuário, foi desenvolvida uma interface em terminal utilizando .NET. Além disso, foi criada uma API REST em ASP.NET Core para permitir a comunicação com outros sistemas. O programa permite consultar clientes pelo código, exibir seus dados cadastrais e atualizar telefone e e-mail. As alterações realizadas são persistidas no arquivo de dados utilizado pelo COBOL. A integração entre .NET e COBOL ocorre por meio de arquivos com estrutura fixa definida por copybooks. Também foram implementados testes automatizados para validar as principais funcionalidades da solução. Dessa forma, o projeto demonstra uma alternativa simples e organizada para integrar tecnologias modernas a sistemas legados.

*Documentação feita como parte da entrega do Trabalho Final do projeto de treinamento AceleraMaker.

¹ Aluno do curso de Ciência da Computação – vitorsetragni@gmail.com.

1 CONTEXTO

A Cooperativa Financeira Alfa utiliza um sistema legado para realizar o cadastro e a manutenção dos dados de seus clientes. Esse sistema é considerado confiável e estável, porém apresenta limitações para integração com aplicações modernas e para uso por novos canais de atendimento.

Diante desse cenário, o projeto teve como objetivo desenvolver uma solução de modernização sem substituir completamente o sistema legado. A proposta foi preservar o processamento em COBOL, responsável pela regra principal de consulta e atualização dos clientes, e criar uma camada moderna em .NET para facilitar o acesso às funcionalidades do sistema.

A solução desenvolvida permite consultar um cliente por meio de seu código, exibir seus dados cadastrais, atualizar telefone e e-mail, informar quando o cliente não é encontrado e persistir as alterações realizadas. Dessa forma, o projeto mantém a compatibilidade com o processamento legado e, ao mesmo tempo, permite que futuras aplicações possam consumir os serviços por meio de uma API.

2 ARQUITETURA DA SOLUÇÃO

A arquitetura da solução foi organizada em três componentes principais: uma interface em terminal, uma API em .NET e um programa COBOL responsável pelo processamento dos dados. O fluxo principal da aplicação pode ser representado da seguinte forma:

`InterfaceTerminal → InterfaceAPI → COBOL → Dados`

A **InterfaceTerminal** é a aplicação utilizada pelo usuário final. Ela apresenta um menu simples no terminal, permitindo pesquisar clientes, atualizar telefone e e-mail ou encerrar o sistema. Essa interface não acessa diretamente os arquivos do COBOL, pois toda comunicação é feita por meio da API.

A **InterfaceAPI** foi desenvolvida em ASP.NET Core Web API e funciona como a camada moderna de integração. Ela recebe as requisições da interface terminal, grava os arquivos de integração utilizados pelo COBOL, executa o programa legado e interpreta o arquivo de resposta gerado. Essa camada também permite que outras aplicações futuramente possam consumir os serviços de cliente.

O componente **COBOL** representa o sistema legado da solução. Ele é responsável por consultar os clientes no arquivo de dados, atualizar telefone e e-mail, informar quando um cliente não é encontrado e persistir as alterações no arquivo `clientes.dat`.

A comunicação entre .NET e COBOL ocorre por meio de arquivos de layout fixo. A API grava uma requisição no arquivo `requisicao.txt`, executa o programa COBOL e depois lê o resultado no arquivo `resposta.txt`. Essa abordagem foi escolhida por ser simples,

compatível com sistemas legados e adequada para demonstrar a integração entre tecnologias modernas e COBOL.

2.1 Fluxo de Execução da Consulta de Cliente

O fluxo completo da consulta pode ser resumido da seguinte forma:

- a) O usuário escolhe a opção 1 - Pesquisar cliente no menu da `InterfaceTerminal`.
- b) A classe `Menu` chama o `ClienteApiService`.
- c) O `ClienteApiService` envia uma requisição GET para `/api/clientes/{codigo}`.
- d) A `InterfaceAPI` recebe a requisição no controller de clientes.
- e) O controller chama a classe `ClienteService`.
- f) O `ClienteService` chama o `CobolClienteGateway`.
- g) O `CobolClienteGateway` grava a operação C e o código do cliente no arquivo `requisicao.txt`.
- h) O `CobolClienteGateway` executa o programa COBOL `Cliente.cob`.
- i) O COBOL lê o arquivo `requisicao.txt`.
- j) O COBOL abre e percorre o arquivo `clientes.dat`.
- k) O COBOL compara o código informado com os códigos dos registros do arquivo.
- l) Se encontrar o cliente, o COBOL grava uma resposta com status OK no arquivo `resposta.txt`.
- m) Se não encontrar o cliente, o COBOL grava uma resposta com status NF.
- n) O `CobolClienteGateway` lê o arquivo `resposta.txt`.
- o) A API retorna os dados do cliente ou uma resposta de cliente não encontrado.
- p) A `InterfaceTerminal` exhibe o resultado para o usuário.

Esse fluxo demonstra a separação de responsabilidades entre as camadas da solução: a interface terminal cuida da interação com o usuário, a API organiza a entrada e saída das requisições, o gateway realiza a integração com o COBOL e o programa COBOL mantém o processamento e a leitura do arquivo legado.

3 TECNOLOGIAS UTILIZADAS E JUSTIFICATIVAS TÉCNICAS

A solução utilizou **.NET** para a construção da interface de usuário e da API. Essa escolha se justifica pela facilidade de criação de aplicações modernas, pela boa organização do código em camadas e pela possibilidade de expor serviços por meio de requisições HTTP.

A **InterfaceTerminal** foi criada como uma aplicação console em .NET. Essa escolha permitiu construir uma interface simples para o usuário digitar o código do cliente, visualizar os dados retornados e informar novos dados de contato. A separação da interface em relação à API facilita a manutenção e evita que a regra de integração fique misturada com a apresentação ao usuário.

A **InterfaceAPI** foi desenvolvida com ASP.NET Core Web API. A API expõe endpoints para consulta e atualização de clientes, funcionando como uma camada intermediária entre a interface e o COBOL. Essa decisão facilita a futura integração com outros sistemas, pois qualquer aplicação poderá consumir os serviços da API sem acessar diretamente o programa COBOL ou os arquivos de dados.

O **COBOL** foi utilizado para representar o componente legado do sistema. Ele ficou responsável pelo processamento principal dos dados, mantendo a ideia de que o sistema antigo continua sendo utilizado, mas agora acessado por uma camada moderna. O programa COBOL consulta e atualiza registros em um arquivo de dados com layout fixo.

Os **copybooks** foram utilizados para definir a estrutura compartilhada entre os componentes. Foram criadas estruturas para cliente, requisição e resposta. Essa decisão ajuda a manter um padrão único de comunicação, evitando divergências entre os dados enviados pela API e os dados esperados pelo programa COBOL.

O arquivo `clientes.dat` foi utilizado como forma de persistência. Cada linha representa um cliente com campos de tamanho fixo, contendo código, nome, telefone e e-mail. Essa abordagem se aproxima do funcionamento de sistemas legados e facilita o processamento pelo COBOL.

O **Swagger** foi mantido na API como ferramenta de apoio para documentação e testes manuais dos endpoints. Embora a interface principal do usuário seja o terminal, o Swagger permite visualizar e testar as rotas da API, contribuindo para a documentação técnica da solução.

Também foi utilizado o **xUnit** para a criação dos testes automatizados. Essa ferramenta permite validar automaticamente as principais funcionalidades da API integrada ao COBOL, reduzindo a necessidade de testes manuais repetitivos e ajudando a identificar falhas após alterações no código.

Por fim, a plataforma **Overleaf** foi utilizada para a escrita e formatação da documentação do projeto. Sua utilização facilitou a organização do texto, permitindo estruturar as seções, tabelas e demais elementos necessários para a entrega final da documentação.

4 TESTES

Os testes da solução foram divididos em testes manuais e testes automatizados. Os testes manuais foram realizados por meio da interface terminal e também pelo Swagger, com o objetivo de validar o comportamento do sistema a partir da perspectiva do usuário e da API.

Na interface terminal, foram testadas as opções de pesquisar cliente, atualizar telefone e e-mail e consultar um cliente inexistente. Também foi verificado se, ao tentar atualizar um cliente que não existe, o sistema informa o erro antes de solicitar os novos dados de contato.

O Swagger foi utilizado como apoio para testar diretamente os endpoints da API. Foram validadas as rotas de consulta e atualização, confirmando que a API recebia as requisições corretamente e retornava as respostas esperadas.

Além dos testes manuais, foram implementados testes automatizados utilizando xUnit. Esses testes verificam o funcionamento da API integrada ao programa COBOL. Ao executar os testes, o sistema prepara os dados iniciais, realiza as chamadas HTTP para a API e valida os resultados retornados.

Foram implementados quatro testes automatizados principais:

- pesquisar cliente existente;
- pesquisar cliente inexistente;
- atualizar telefone e e-mail de cliente existente;
- tentar atualizar cliente inexistente.

O teste de atualização de cliente existente também verifica a persistência dos dados, pois após atualizar telefone e e-mail, uma nova consulta é realizada para confirmar se as alterações foram realmente salvas no arquivo utilizado pelo COBOL.

Dessa forma, os testes cobrem as principais funcionalidades exigidas pela solução: consulta, atualização, tratamento de cliente inexistente e persistência dos dados. Como os testes automatizados acessam a API e a API executa o COBOL, eles também validam a integração entre os componentes principais do projeto.

5 USO DE INTELIGÊNCIA ARTIFICIAL

Tabela 1 – Registro de uso de IA no trabalho

Prompt utilizado	Finalidade	Validação humana
"Me indique materiais que falam sobre a conexão entre .NET e COBOL."	Obter indicações de conteúdos para estudar a integração entre aplicações modernas em .NET e sistemas legados em COBOL.	Os materiais indicados foram utilizados apenas como apoio de estudo, sendo comparados com os requisitos do projeto antes da definição da arquitetura.
"Quais ferramentas me permitem testar de forma automática meu código que faz integração .NET e COBOL? Me mande indicações de materiais de estudo de cada uma delas."	Identificar ferramentas adequadas para a criação de testes automatizados e obter materiais de estudo sobre cada alternativa.	A ferramenta escolhida foi validada na prática por meio da implementação e execução dos testes automatizados
"Revise a ortografia da minha documentação e aponte todas as mudanças feitas, mesmo que seja o erro de uma letra."	Apoiar a revisão textual da documentação, corrigindo erros de escrita e melhorando a clareza do texto.	O texto revisado foi conferido manualmente para garantir que nenhuma informação técnica incorreta ou não implementada fosse adicionada.